

Лабораторная работа №3. Работа с Markdown-разметкой и базовое использование LaTeX в документации проекта

Markdown — это лёгкий язык разметки, который используется для оформления README-файлов, документации, отчётов и даже научных текстов. Он прост в использовании, хорошо читается как в сыром виде, так и после рендеринга, и широко поддерживается GitHub, VS Code и многими платформами.

LaTeX — мощная система вёрстки, которая позволяет оформлять формулы, математические выражения и структурированные документы прямо внутри Markdown-файлов (например, в GitHub или Obsidian).

Шаг 1. Создание рабочей директории

1. Откройте **Visual Studio Code**. У вас по умолчанию откроется предыдущий проект, поэтому выберите **File** → **New Window**.

2. Нажмите **Ctrl + `** чтобы открыть терминал VS Code.

3. Перейдите в папку с документами командой:

cd Documents/

```
Paltos@Paltosik92 MINGW64 ~  
$ cd Documents/  
  
Paltos@Paltosik92 MINGW64 ~/Documents
```

4. Создайте директорию для лабораторной работы:

Lab3_MarkdownLaTeX_FIO

(где *FIO* — ваши фамилия).

```
Paltos@Paltosik92 MINGW64 ~/Documents  
$ mkdir Lab3_MarkdownLaTeX_Leontev
```

5. Убедитесь, что папка создана:

```
Paltos@Paltosik92 MINGW64 ~/Documents  
$ ls  
Blender/                               Lab3_MarkdownLaTeX_Leontev/
```

6. Перейдите в созданную директорию:

```
Paltos@Paltosik92 MINGW64 ~/Documents  
$ cd Lab3_MarkdownLaTeX_Leontev  
  
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev
```

7. Создайте подкаталоги:

- **docs** — для Markdown-файлов
- **img** — для скриншотов
- **latex** — для примеров формул

Рекомендация - в *Git Bash* (и вообще в *Bash*) вы можете создать несколько директорий одной командой *mkdir*, для этого используйте пробелы между названиями директорий:

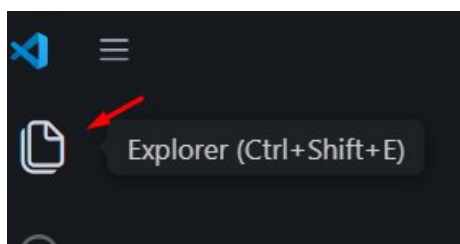
```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev
$ mkdir docs img latex
```

Проверим создание:

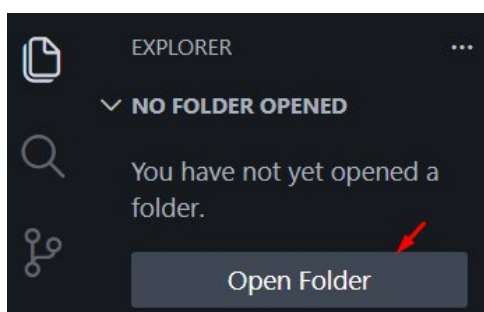
```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev
$ ls
docs/  img/  latex/
```

Шаг 2. Подключаем рабочий каталог в VS Code

1. Откройте **Explorer** (сочетание клавиш *Ctrl + B*).

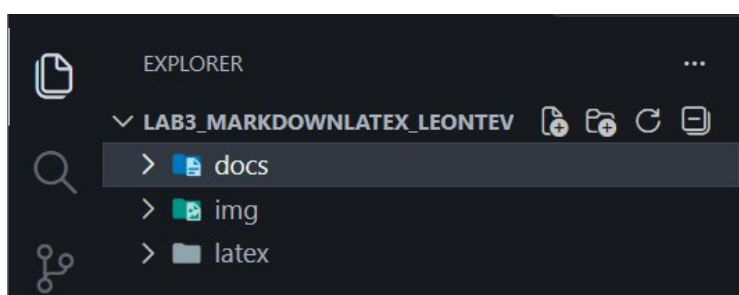


2. Нажмите **Open Folder**.



3. Выберите папку **Lab3_MarkdownLaTeX_FIO** и откройте её.

Теперь VS Code работает внутри вашего проекта.



Шаг 3. Инициализация Git-репозитория

Возвращаемся в терминал (если закрыли **Ctrl + `**).

1. Инициализируйте репозиторий:

git init

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev
$ git init
Initialized empty Git repository in C:/Users/Paltos/Documents/Lab3_MarkdownLaTeX_Leontev/.git/
```

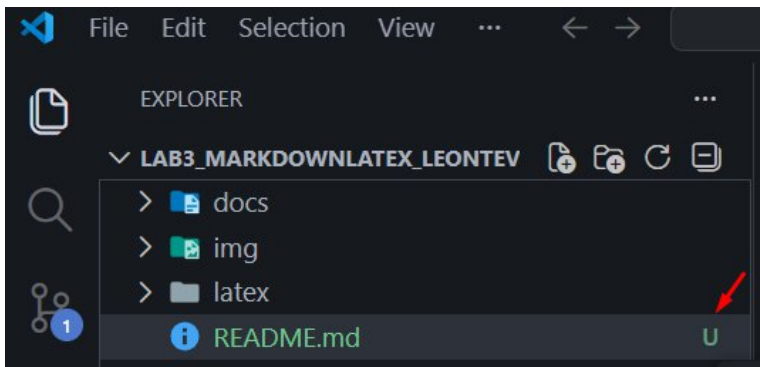
2. Создайте первый файл и проверьте его создание:

touch README.md

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev (master)
$ touch README.md

Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev (master)
$ ls
docs/  img/  latex/  README.md
```

Обратите внимание файл также отображается в **Explorer**:



Он имеет состояние “**Untracked**” (не отслеживаемый). Мы можем в этом убедиться, введя команду:

git status

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  README.md

nothing added to commit but untracked files present (use "git add" to track)
```

3. Добавьте файл в индекс и снова проверим его статус:

git add README.md

git status

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev
$ git add README.md

Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md
```

Мы видим, что файл готов для коммита.

4. Создайте коммит:

git commit -m "Initial commit: created README.md for Lab3"

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev (master)
$ git commit -m "Initial commit: created README.md for Lab3"
[master (root-commit) cf9ffd5] Initial commit: created README.md for Lab3
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
```

Информация о коммите:

- **Хеш коммита:** cf9ffd5 (сокращенный, полный обычно 40 символов)
- **Ветка:** master (основная ветка)
- **Особенность:** (root-commit) — это самый первый коммит в репозитории
- **Сообщение:** Initial commit: created README.md for Lab3
 - **Изменения:**
- **Файлы:** 1 файл изменен
- **Тип изменений:** create mode 100644 README.md
 - create — файл создан (неизменен)
 - mode 100644 — стандартные права доступа для обычного файла
 - README.md — созданный файл
- **Статистика:**
- **Вставки (+):** 0 строк добавлено
- **Удаления (-):** 0 строк удалено
 - **Факт:** Файл создан, но пока пустой (без содержимого)

Шаг 4. Подключение к GitHub

1. Подготовьте репозиторий на GitHub:

- Создайте новый пустой **public**-репозиторий:
Lab3-MarkdownLaTeX_FIO
- Без README, .gitignore и License.

1 General

Owner * Paltosik92

Repository name * Lab3-MarkdownLaTeX_Leontev

✓ Lab3-MarkdownLaTeX_Leontev is available.

Great repository names are short and memorable. How about [bookish-octo-sniffle?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility * Public

Choose who can see and commit to this repository

Add README Off

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license

Licenses explain how others can use your code. [About licenses](#)

Create repository

2. Привяжите локальный репозиторий:

git remote add origin <URL_вашего_репозитория>

git push -u origin master

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev (master)
$ git remote add origin https://github.com/Paltosik92/Lab3-MarkdownLaTeX_Leontev.git

Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 242 bytes | 242.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Paltosik92/Lab3-MarkdownLaTeX_Leontev.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Краткий разбор пуша:

1. Подсчет объектов (Enumerating objects):

- Найдено: 3 объекта Git

- **Обычно это:** 1 blob (файл README.md) + 1 tree (структура) + 1 commit (сам коммит)

2. Сжатие и подготовка (Counting objects):

- **Прогресс:** 100% (3/3)
- Git сжимает и подготавливает объекты для отправки

3. Отправка данных (Writing objects):

- **Размер:** 242 байта (очень мало — пустой файл)
- **Скорость:** 242.00 KiB/s

4. Статистика (Total 3):

- **Всего объектов:** 3
- **Delta 0:** Нет дельта-компрессии (первый пуш, нечего сравнивать)
- **Reused 0:** Ничего не переиспользовано
- **Pack-reused 0:** Нет переиспользованных pack-файлов
 - **Информация о репозитории:**
- **Удаленный репозиторий:** https://github.com/Paltosik92/Lab3-MarkdownLaTeX_Leontev.git
- **Ветка:** master
 - **Результат пуша:**
- * [new branch] master -> master — создана новая ветка master на удаленном репозитории
- branch 'master' set up to track 'origin/master' — локальная ветка master настроена на отслеживание удаленной origin/master

Практическое задание 1

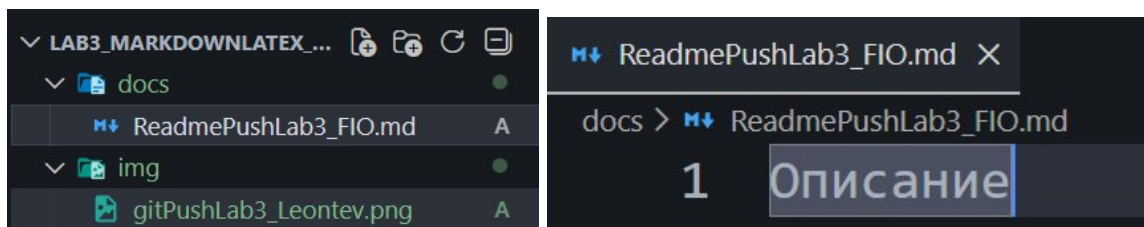
1. Сделайте скриншот Git Bash с командами привязки репозитория и пушем.

Сохраните его в **img** под названием:

gitPushLab3_FIO.png

2. В папке **docs** создайте Markdown-файл с кратким описанием вашего пуша:

ReadmePushLab3_FIO.md



3. Очистите терминал командой:

clear

4. Добавьте все файлы проекта и создайте коммит:

git add .

git commit -m "Added project structure and push description"

5. Сделайте скриншот после коммита. Назовите файл:

commitStructureLab3_FIO.png

и сохраните в **img**.

6. Добавьте все файлы проекта:

git add .

7. Создайте коммит:

git commit -m "Added commit image"

8. Запушьте изменения:

git push

Шаг 5. Заголовки и подзаголовки в Markdown

В Markdown заголовки создаются с помощью символа **#**.

Количество **#** определяет уровень заголовка:

Уровень	Синтаксис	Рендерится как
1	# Заголовок	<h1>
2	## Подзаголовок	<h2>
3	### Подраздел	<h3>
4	####	Мелкие заголовки

Использование заголовков помогает структурировать документацию, разбивать текст на разделы и улучшать читаемость README-файлов.

Редактирование Markdown-файла

1. Перейдите в папку **docs/**.

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev (master)
$ cd docs

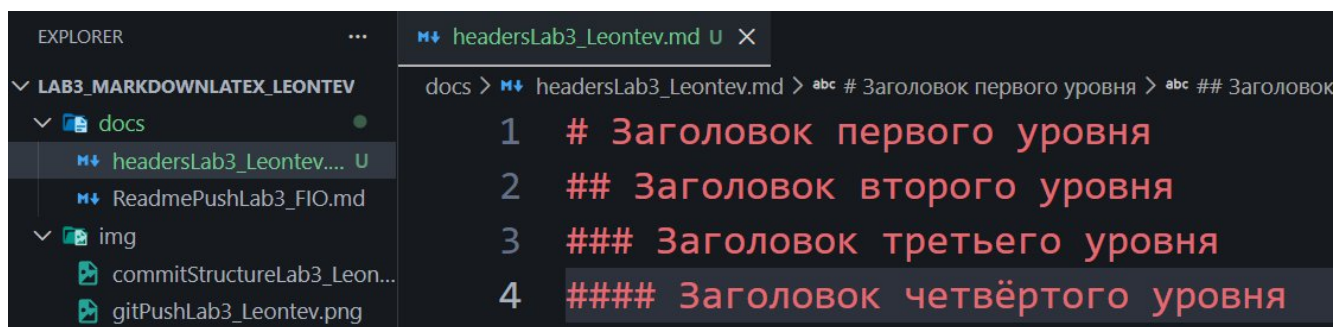
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev/docs (master)
```

2. Создайте новый файл:

touch headersLab3_FIO.md

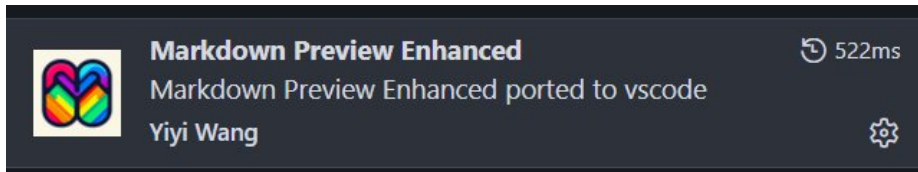
```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev/docs (master)
$ touch headersLab3_Leontev.md
```

3. Откройте файл в VS Code и добавьте примеры всех уровней заголовков:



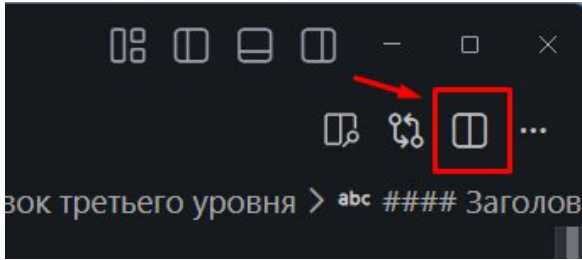
4. Установите расширение для удобного просмотра Markdown в VS Code:

- Откройте панель расширений (Ctrl+Shift+X)
- Найдите и установите **Markdown Preview Enhanced**



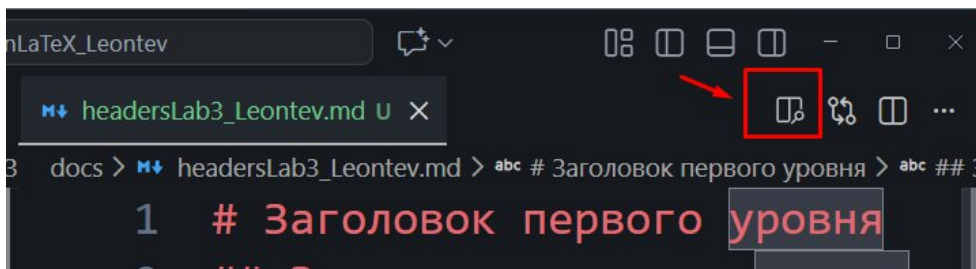
5. Настройте параллельный просмотр в VS Code:

- Нажмите на иконку **Split Editor** (разделить редактор)

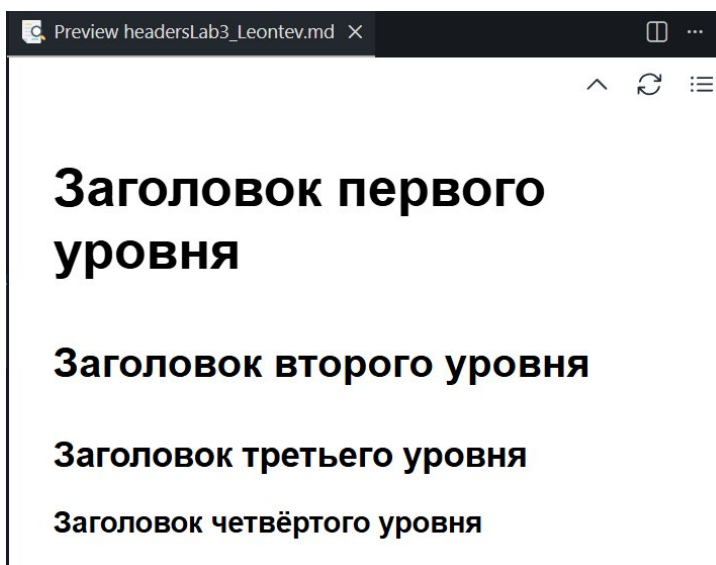


- В правом окне откройте предпросмотр через:

- Правой кнопкой → **Open Preview**
- Или **Ctrl+Shift+V**



Теперь изменения в Markdown будут отображаться в реальном времени в соседнем окне:



Практическое задание 2

1. Добавьте в файл **headersLab3_FIO.md**:

- название вашей лабораторной работы как заголовков уровня H2;
- 3 подзаголовка уровня H4, где кратко опишите:
 - что такое Markdown,
 - где он применяется,
 - зачем он нужен разработчикам.

Пример реализации (переписывать не обязательно, можете найти необходимую информацию в поисковых системах):

Лабораторная работа №3. Работа с Markdown-разметкой и базовое использование LaTeX в документации проекта

Markdown — это лёгкий язык разметки, который используется для оформления README-файлов, документации, отчётов и даже научных текстов. Он прост в использовании, хорошо читается как в сыром виде, так и после рендеринга, и широко поддерживается GitHub, VS Code и многими платформами.

Markdown находит широкое применение в различных сферах:

Веб-разработка.
Блоги и платформы для написания.
Электронные письма и форумы.
Статические сайты.
Документация.
Системы управления версиями.
Примечания и заметки.
Презентации.

Разработчикам Markdown нужен для быстрого и простого написания документации, README-файлов, заметок, комментариев и сообщений, которые легко читать и форматировать в простом тексте, а затем преобразовывать в HTML, PDF или другие форматы без сложных редакторов. Он экономит время, повышает читабельность и обеспечивает единообразное

2. Выполните команды:

git add .

git commit -m "Added Markdown headers examples and explanations"

git push

Обратите внимание вы должны будете сперва подняться на уровень выше командой `cd ..` во всех следующих примера.

3. Сделайте скриншот терминала коммита и пуша.

Сохраните как **headersCommitLab3_FIO.png** в папку *img/*.

Шаг 6. Горизонтальные линии

Горизонтальные разделители используются для выделения крупных частей документа. Они создаются тремя и более символами:

- ---
- ***
- ____

Все варианты работают одинаково и визуально отделяют содержимое.

Редактирование Markdown-файла

1. Создайте новый файл в папке **docs**:

touch separatorsLab3_FIO.md

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev/docs (master)
$ touch separatorsLab3_Leontev.md
```

2. Вставьте примеры разделителей:

```
1 ~ Первый раздел
2 ---
3 Второй раздел
4 ***
5 Третий раздел
6 ____
```

Первый раздел

Второй раздел

Третий раздел

Практическое задание 3

1. Добавьте в файл **separatorsLab3_FIO.md** три смысловых раздела:
 - "Введение"
 - "Основные возможности Markdown"
 - "Пример форматирования"

Каждый раздел должен быть отделён горизонтальной линией. **Пример реализации** (переписывать не обязательно, можете найти необходимую информацию в поисковых системах):

Markdown — это простой язык разметки для быстрого форматирования текста (заголовки, списки, жирный/курсив), который легко преобразуется в HTML и используется для создания документации, постов в блогах и веб-контента, позволяя сосредоточиться на тексте, а не на сложном коде.

Основные возможности Markdown — это быстрое и простое форматирование текста (заголовки, списки, ссылки, выделение), сохраняющее читабельность, для последующего преобразования в HTML или другие форматы, идеальное для документации, заметок, README-файлов, блогов и сообщений, с синтаксисом, использующим обычные символы вместо сложного кода, что делает его удобным и интуитивно понятным.

Пример форматирования. Заголовки:

Заголовок 1

Заголовок 2

Заголовок 3

2. Сделайте коммит и пуш:

git add .

git commit -m "Added horizontal separators examples and structured sections"

git push

3. Сделайте скриншот коммита и пуша:

separatorsCommitLab3_FIO.png

Сохраните в папку *img/*.

Шаг 7. Текстовое форматирование

Markdown поддерживает простое визуальное оформление текста:

Формат	Синтаксис	Пример
Жирный текст	**текст**	текст
<i>Курсив</i>	<i>*текст*</i>	<i>текст</i>
<i>Жирный курсив</i>	<i>***текст***</i>	<i>текст</i>
<u>Зачёркнутый</u>	<u>~текст~</u>	<u>текст</u>
Код внутри текста	<code>`код`</code>	код
Блок кода	тройные бэктики	<code>console.log("hi")</code>

Форматирование помогает выделять термины, команды, фрагменты кода и важные части текста.

Создание и редактирование файла

1. Создайте файл:

touch formattingLab3_FIO.md

2. Добавьте примеры всех типов форматирования.

```

**Жирный текст**
*Курсив*
***Жирный курсив***
~~Зачёркнутый~~
`Console.WriteLine("Hello");`
```
string name;
name = Console.ReadLine();
Console.WriteLine(name)
```

```

Жирный текст

Курсив

Жирный курсив

Зачёркнутый

```
Console.WriteLine("Hello");
```

```
string name;
name = Console.ReadLine();
Console.WriteLine(name)
```

Рекомендация: если в начале блока кода ввести язык программирования, то если ваш редактор поддерживает синтаксис, у вас будут отображаться корректно цвета вашего языка:

```

```csharp
string name;
name = Console.ReadLine();
Console.WriteLine();
```

```

Практическое задание 4

Оформите следующее задание по образцу (пример ниже)

Задание: Комментированная программа на C#

Создайте консольное приложение на C#, которое демонстрирует все форматы Markdown в комментариях к коду.

Требования к программе:

Имя файла: FormatDemo.cs

Логика:

Запрашивает у пользователя два числа;

Выполняет их сложение;

Выводит результаты в форматированном виде.

Пример реализации:

Задание: Комментированная программа на C#

Создайте консольное приложение на C#, которое демонстрирует все форматы Markdown в комментариях к коду.

Требования к программе:

1. **Имя файла:** `FormatDemo.cs`
2. **Логика:**
 - Запрашивает у пользователя **два числа**
 - Выполняет **их сложение**
 - Выводит результаты в **форматированном виде**

Пример структуры кода с комментариями в стиле Markdown:

```
Console.Write("Введите первое число: ");
double number1 = Convert.ToDouble(Console.ReadLine());
Console.Write("Введите второе число: ");
double number2 = Convert.ToDouble(Console.ReadLine());
double sum = number1 + number2;
Console.WriteLine($"**Результаты операций: {sum}**");
```

Зафиксируйте изменения:

git add .

git commit -m "Added text formatting examples and explanations"

git push

Сделайте скриншот с коммита и пуша:

formattingCommitLab3_FIO.png

Шаг 8. Списки в Markdown

Списки помогают структурировать информацию, перечислять пункты и облегчать восприятие текста.

Markdown поддерживает два вида списков:

Маркированные списки

Создаются с помощью символов:

- 1
- 2
- 3

Нумерованные списки

Создаются с помощью:

1. Первый пункт
2. Второй пункт
3. Третий пункт

Вложенные списки

Просто используйте отступ (таб или 4 пробела):

1. Пункт
 - Подпункт
 - Подпункт

Создание и редактирование файла

Создайте файл в папке **docs**:

touch listsLab3_FIO.md

Добавьте примеры всех видов списков.

Практическое задание 5

У вас есть следующий текст:

План разработки

Задачи на сегодня

Изучить систему контроля версий

- Git базовые команды
- Работа с ветками
- Устаревший подход Современный workflow

Освоить разметку документации

- Markdown синтаксис
- Форматирование текста
- Создание таблиц и списков

Изучить основы программирования

- JavaScript синтаксис
- Работа с DOM
- Асинхронные операции

Как создать репозиторий

Маркированные списки

Создаются с помощью символов:

- *
- -
- +

Нумерованные списки

Создаются с помощью:

1. Первый пункт
2. Второй пункт
3. Третий пункт

Вложенные списки

Просто используйте отступ (таб или 4 пробела):

1. Пункт
 - Подпункт
 - Подпункт

Подготовка структуры

Создать папку проекта со следующей структурой:

Файл *README.md* для описания проекта

Папка *src/* для хранения исходного кода

Папка *docs/* для документации

Файл *.gitignore* для исключения ненужных файлов

Настройка репозитория

Инициализировать Git-репозиторий

`git init`

Настройка репозитория

Добавить все файлы в staging area

`git add .`

Проверить статус изменений

`git status`

Создать первый коммит с описанием

`git commit -m "Initial commit: базовая структура проекта"`

Работа с удалённым репозиторием

Настройка удаленного репозитория

`git remote add origin <url>`

`git push -u origin main`

Вам нужно его оформить по следующему примеру (**текст на одну страницу**):

План разработки

Задачи на сегодня

1. Изучить систему контроля версий

- Git базовые команды
- Работа с ветками
- Устаревший подход Современный workflow

2. Освоить разметку документации

- Markdown синтаксис
- Форматирование текста
- Создание таблиц и списков

3. Изучить основы программирования

- JavaScript синтаксис
- Работа с DOM
- Асинхронные операции

Как создать репозиторий

Шаг 1: Подготовка структуры

1. Создать папку проекта со следующей структурой:

- Файл *README.md* для описания проекта
- Папка *src/* для хранения исходного кода
- Папка *docs/* для документации
- Файл *.gitignore* для исключения ненужных файлов

Шаг 2: Настройка репозитория

2. Инициализировать Git-репозиторий

`git init`

3. Настройка репозитория

Добавить все файлы в staging area

`git add .`

4. Проверить статус изменений

`git status`

5. Создать первый коммит с описанием

`git commit -m "Initial commit: базовая структура проекта"`

Шаг 3: Работа с удалённым репозиторием

4. Настройка удаленного репозитория

```
git remote add origin <url>
git push -u origin main
```

2. Сделайте коммит и пуш:

`git add .`

`git commit -m "Added examples of bullet, numbered and nested lists"`

`git push`

3. Сделайте скриншот коммита и пуша, сохраните как **listsCommitLab3_FIO.png** в папку *img/*.

Шаг 9. Ссылки и изображения в Markdown

Markdown позволяет легко вставлять ссылки и изображения.

Ссылки

Обычные ссылки:

[Текст ссылки](https://example.com)

Ссылки с подсказкой:

[GitHub](https://github.com "Перейти на GitHub")

Изображения

Синтаксис похож на ссылки, но начинается с !:

![Описание изображения](путь/к/файлу.png)

Для локальных изображений путь указывается относительно .md файла.

Создание и редактирование файла

Создайте файл:

touch linksImagesLab3_FIO.md

Добавьте внутри:

- 1 ссылку на GitHub,
- 1 ссылку на официальный сайт Markdown с подсказкой,
- 1 изображение из вашей папки *img/*.

Пример выполнения:

```
[Github] (https://github.com/Paltosik92/Lab3-MarkdownLaTeX_Leontev)
[Markdown] (https://www.markdownguide.org/"Перейти на официальный сайт")
![Скриншот задания](/img/commitStructureLab3_Leontev.png)
```

```
[Github] (https://github.com/Paltosik92/Lab3-MarkdownLaTeX_Leontev)
[Markdown] (https://www.markdownguide.org/"Перейти на официальный сайт")
Paltos@Paltosik92 MINGW64 ~/Documents/Lab3_MarkdownLaTeX_Leontev (master)
$ git commit -m "Added project structure and push description"
[master e59dc56] Added project structure and push description
2 files changed, 1 insertion(+)
create mode 100644 docs/ReadmePushLab3_FIO.md
create mode 100644 img/gitPushLab3_Leontev.png
```

Практическое задание 6

1. Добавьте в файл **linksImagesLab3_FIO.md**:
 - 3 внешние ссылки,
 - 2 изображения локально,
2. Сделайте коммит и пуш:

git add .

git commit -m "Added examples of links and images in Markdown"

git push

3. Сделайте скриншот коммита и пуша **linksCommitLab3_FIO.png** и сохраните в папке *img/*.

Шаг 10. Блоки кода и цитаты

Markdown позволяет вставлять фрагменты кода с подсветкой синтаксиса, а также цитаты.

Блоки кода

Встроенный (inline) код:

Команда ``git status`` показывает состояние репозитория.

Многострочный код:

```
```python
print("Hello, Markdown!")
```
```

Можно указать язык: `python`, `js`, `bash`, `csharp` и др.

Цитаты

Цитата создаётся символом `>`:

`>` Это цитата в Markdown.

Вложенная цитата:

`>` Первый уровень

`>>` Второй уровень

Создание и редактирование файла

Создайте файл:

touch codeQuotesLab3_FIO.md

Добавьте примеры:

- inline-кода,
- блоков кода с подсветкой,
- обычных блоков кода,
- цитат.

Блоки кода

Встроенный (inline) код:

Команда `git status` показывает состояние репозитория.

Многострочный код:

```
print("Hello, Markdown!")
```

Можно указать язык: python, js, bash, csharp и др.

Цитаты

Цитата создаётся символом >:

Это цитата в Markdown.

Вложенная цитата:

Первый уровень

Второй уровень

Практическое задание 7

1. В файл **codeQuotesLab3_FIO.md** добавьте:

- пример команды Git в виде inline-кода;
- блок кода на любом языке программирования;
- блок кода без указания языка;
- две цитаты: обычную и вложенную.

2. Выполните коммит:

git add .

git commit -m "Added code blocks, inline code and quotes examples"

git push

3. Сохраните скриншот коммита и пуша **codeCommitLab3_FIO.png** в папке *img/*

Шаг 11. Таблицы в Markdown

Таблицы в Markdown позволяют структурированно оформлять данные. Они особенно полезны в отчётах, инструкциях и README-документах.

Синтаксис таблиц

Простейшая таблица:

| Столбец 1 | Столбец 2 |

|-----|-----|

| Значение | Значение |

Выравнивание столбцов:

| Лево | Центр | Право |

|:-----|:-----|:-----|

| a | b | c |

- `:-` — выравнивание по левому краю
- `:---` — по центру
- `---` — по правому краю

Выравнивание улучшает читаемость таблиц, особенно с числами или длинными текстами.

Создание и редактирование файла

Создайте файл:

`touch tablesLab3_FIO.md`

Практическое задание 8

1. В файл **`tablesLab3_FIO.md`** добавьте таблицу:

| Функция | Команда Git | Описание |
|----------------------|--------------------------------|-----------------------------|
| Создание репозитория | <code>git init</code> | Инициализация репозитория |
| Просмотр состояния | <code>git status</code> | Показывает изменённые файлы |
| Отправка на GitHub | <code>git push</code> | Загружает коммиты на сервер |

2. Сделайте её с выравниваем трёх столбцов

3. Зафиксируйте изменения:

`git add .`

`git commit -m "Added Markdown tables with alignment and Git examples"`

`git push`

4. Сохраните скриншот коммита и пуша **`tablesCommitLab3_FIO.png`** в папке *img/*

Шаг 12. Дополнительные возможности Markdown

Помимо базовых средств, Markdown поддерживает расширенные элементы, которые активно используются в GitHub-проектах, документации и README-файлах. Эти инструменты не обязательны, но значительно улучшают структуру и выразительность документа.

12.1. Чекбоксы (списки задач)

Используются в документации, планировании задач и issue-трекерах на GitHub.

- [] Задача не выполнена

- [x] Задача выполнена

Чекбоксы отображаются как интерактивные элементы на GitHub.

12.2. Сноски (footnotes)

Работают на GitHub и позволяют добавлять дополнительные пояснения:

Текст со сноской^[^1].

[^1]: Это текст сноски.

12.3. GitHub Alert-блоки

GitHub поддерживает специальные форматированные блоки:

> [!NOTE]

> Это простая заметка.

> [!TIP]

> Полезный совет.

> [!IMPORTANT]

> Важная информация.

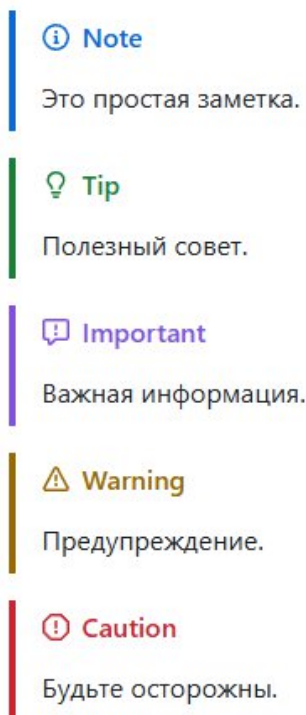
> [!WARNING]

> Предупреждение.

> [!CAUTION]

> Будьте осторожны.

В VS Code они не отображаются, но при публикации на Github выглядят следующим образом:



12.4. Escape-символы (экранирование)

Если необходимо вывести символ, который Markdown воспринимает как управляющий:

\# Это не заголовок

это просто звёздочка

12.5. Встроенный HTML

Markdown поддерживает HTML-теги, что позволяет расширить возможности разметки:

`<p style="color:red;">Красный текст</p>`

`<b>Жирный HTML-текст</b>`

Например, таблицы с `colspan` или управляемые абзацы удобнее писать на HTML.

Создание и редактирование файла

Создайте новый Markdown-файл:

touch advancedMarkdownLab3_FIO.md

Практическое задание 9

1. В файл **advancedMarkdownLab3_FIO.md** добавьте:

- **ЧЕКБОКСЫ**

Мини-план на день (3–4 пункта)

- **СНОСКУ**

Например: Markdown полезен в разработке[^markdown].

и саму сноску снизу.

- **ALERT-БЛОК**

Используйте один блок TIP или IMPORTANT.

- **ESCAPE-СИМВОЛЫ**

Например, экранируйте #, * или _.

- **HTML**

Используйте один HTML-тег с атрибутом (например, `<p>`).

Пример выполнения:

Пример выполнения:

Расширенный Markdown: Дополнительные возможности

Чекбоксы

Мини-план на день

- ☐ Изучить продвинутые возможности Markdown
- ☒ Практиковать создание таблиц и списков
- ☐ Реализовать задание по программированию
- ☐ Отложить Завершить оформление документации

Сноска

Markdown полезен в разработке^[1].

Alert-блок

[!TIP]

Совет по использованию: Для экранирования специальных символов в Markdown используйте обратный слэш \. Например, чтобы написать **курсив** без форматирования, напишите *курсив*.

Escape-символы

- Заголовок без форматирования: \# Это не заголовок
- Звёздочка как текст: *Просто звёздочка*

HTML в Markdown

```
<p style="background-color: #f8f9fa; padding: 15px; border-radius: 5px;">
  Этот параграф оформлен с помощью <strong>HTML-тега</strong> внутри Markdown-документа.
</p>
```

1. Синтаксис Markdown поддерживается на GitHub, GitLab, в большинстве редакторов кода и систем документирования. [↩](#)

2. Выполните коммит:

git add .

git commit -m "Added advanced Markdown examples: checkboxes, footnotes, alerts, escapes, HTML"

git push

3. Сделайте скриншот коммита и пуша. Сохраните его как: **advancedMarkdownCommitLab3_FIO.png** в папку *img/*.

Шаг 13. LaTeX в Markdown

Markdown поддерживает использование математических формул с помощью синтаксиса LaTeX. Эта возможность особенно полезна при оформлении отчётов, научных текстов, документации по алгоритмам и математическим моделям.

GitHub поддерживает рендеринг LaTeX, начиная с 2022 года, в виде:

- **Inline-формул**
- **Блочных формул**

13.1. Inline-формулы

Используются для небольших выражений внутри строки.

Оформляются в $\$ \dots \$$.

Пример:

Площадь круга вычисляется по формуле $S = \pi r^2$.

13.2. Блочные формулы

Используются для больших выражений и выделяются визуально.

Оформляются в $\$ \$ \dots \$ \$$.

Пример:

$$\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$

13.3. Часто используемые конструкции LaTeX

Степени и индексы

x^2 , a_i , x^{10}

Дроби

$\frac{a}{b}$

Корни

$\sqrt{x}$, $\sqrt[3]{y}$

Греческие буквы

α , β , γ , λ

Суммы и произведения

$\sum_{i=1}^n i$

$\prod_{k=1}^n k$

Матрицы

$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

Создание и редактирование файла

Создайте файл:

touch latexLab3_FIO.md

Добавьте в файл все вышеперечисленные примеры.

Практическое задание 10

1. Добавьте в **latexLab3_FIO.md**:

Inline-формулу

Например:

$$E = mc^2$$

Блочную формулу

Например:

интеграл, сумма, матрица или что-то другое.

Три конструкции из списка:

- дробь `\frac{a}{b}`
- корень
- греческие буквы
- степень
- индекс

2. Сделайте коммит:

git add .

git commit -m "Added LaTeX examples: inline, block and basic math constructs"

git push

3. Сохраните скриншот коммита и пуша **latexCommitLab3_FIO.png** в папку *img/*.

Самостоятельное задание: оформление основного README.md

В начале лабораторной работы вы создали файл **README.md** в корневой директории проекта.

Теперь ваша задача — оформить этот файл, используя все навыки, полученные в ходе лабораторной.

Это задание моделирует реальную ситуацию: грамотное оформление README — важнейшая часть любого проекта на GitHub.

15.1. Требования к оформлению README.md

Откройте файл:

README.md

Заполните его, добавив следующие элементы.

Обязательные блоки:

1. **Заголовок уровня H1** — название проекта
2. **Краткое описание** — 2–4 предложения о цели лабораторной
3. **Содержание (Table of Contents)**
 - Ссылки на разделы внутри документа
4. **Структура проекта**
 - списки
 - вложенные списки

Markdown-элементы, которые обязательно должны быть в файле

Добавьте примеры следующих возможностей:

1. Заголовки (H1–H3)

2. Горизонтальная линия

3. Форматирование текста

- **полужирный**
- *курсив*
- **зачёркнутый**
- моноширный

4. Списки

- маркированный
- нумерованный
- вложенный

5. Цитата

6. Блок кода

с указанием языка

7. Таблица

минимум на 3 столбца

8. Изображение из папки *img/*

Пример:

![Push screenshot](./img/gitPushLab3_FIO.png)

9. Ссылка

внешняя и внутренняя

10. Чекбоксы

- Task 1
- Task 2

11. Сноска

В стиле GitHub

Текст^[1]

^[1]: Примечание

12. Alert-блоки GitHub

- [!NOTE]
- [!TIP]
- [!WARNING]

13. Inline LaTeX

Пример:

Площадь круга: $S = \pi r^2$

14. Block LaTeX

Пример:

\$\$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

\$\$

15.2. Финальный коммит

После заполнения **README.md**:

git add .

git commit -m "Completed README.md with full Markdown and LaTeX showcase"

git push

Сделайте скриншот коммита и пуша **readmeFinalPushLab3_FIO.png** в папку *img/*.

15.3. Итог

После выполнения этого задания ваш **README** будет выглядеть как профессиональная страница проекта на **GitHub** — полноформатная, структурированная и демонстрирующая владение **Markdown** и **LaTeX**.